

Digital Twin of Husky

Final Presentation

Manu Benny (244058)
Ijaaz Muhammed Mullahamangalam (244754)
Atif Harshad (244570)

Autonomous Multisensor Systems Group
Institute for Intelligent Cooperating Systems
Faculty of Computer Science
Otto von Guericke University Magdeburg

24.03.2025

- Objective
- Project Overview
- Development Process
- Top Structure Integration
- Sensor Integration:
 - Livox HAP LiDAR
 - Stereolabs ZED2i Camera
 - Emlid Reach RS2+ GPS
- Live Demonstration
- Challenges
- Conclusion and Future Work



<https://clearpathrobotics.com/husky-a300-unmanned-ground-vehicle-robot/>

Goal:

- Create a digital twin of the Husky robot using ROS2 Foxy to test perception and navigation in a simulated environment, meeting the requirement of a ROS Gazebo Simulation.

Specific Objectives:

- Simulated environment and perception system. Leverage ROS programming skills (Python), XACRO (XML based) for robot description.

Why Simulation?

- Cost-effective, safe, and scalable alternative to physical testing.

Target Environment:

- Ubuntu 20.04, ROS2 Foxy, Gazebo, RViz.



Visual Studio Code



GAZEBO

Components:

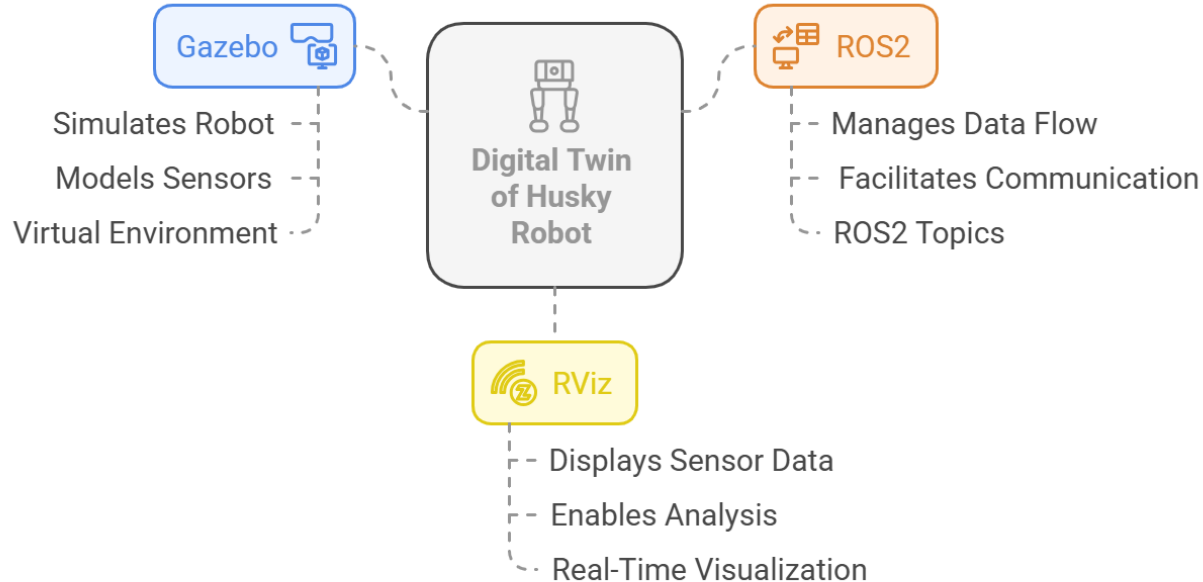
- Husky Robot (base platform).
- Sensors:
 - Livox HAP LiDAR (3D point clouds)
 - Stereolabs ZED2i Camera (stereo images)
 - Emlid Reach RS2+ GPS (position tracking)

Workflow:

- Gazebo simulates the environment and sensors
- Sensor plugins publish data to ROS2 topics
- RViz visualizes the data for analysis



Digital Twin of Husky Robot: Components and Data Flow



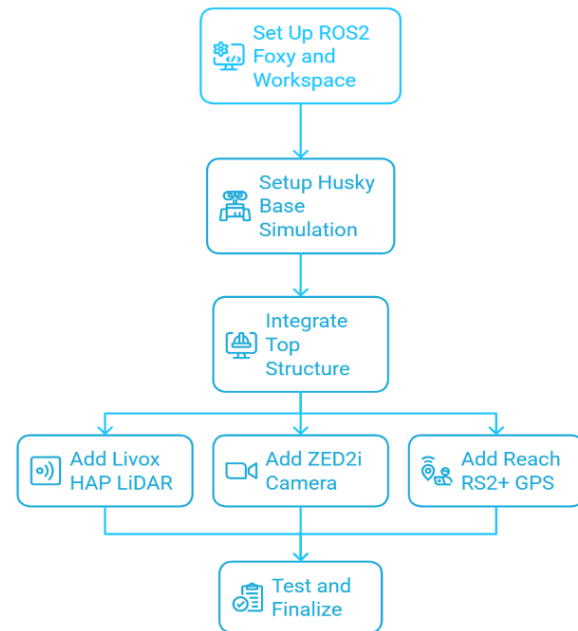
AI generated image -<https://app.napkin.ai/>

Procedural Steps

- Set up ROS2 Foxy and workspace.
- Integrated the Husky base simulation.
- Integrated the custom top structure for sensor mounting.
- Added Livox HAP LiDAR, ZED2i Camera, and Reach RS2+ GPS.
- Conducted testing and finalization.

Project Management:

- Defined SMART goals and a project timeline at the beginning.
- Held regular meetings and presented milestones, aligning with the expected procedure.



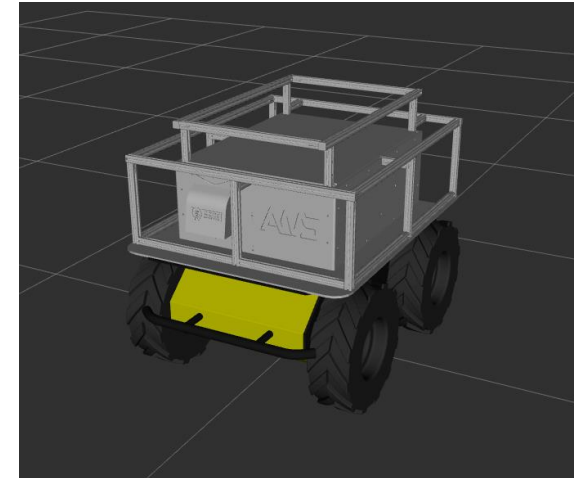
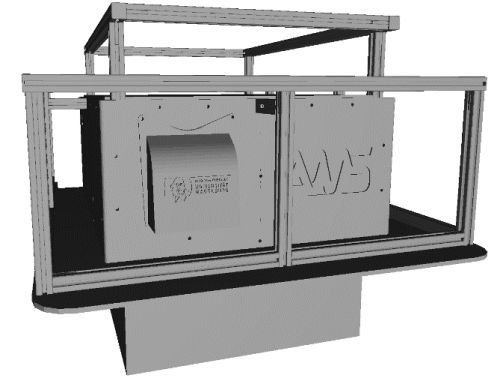
Purpose:

The designed custom top plate (**top_structure**) was used to mount all the three sensors on the Husky robot.

Implementation:

- The designed 3D model (**base_link1.STL**) was added to /meshes
- The definition of (**top_structure**) was integrated to **husky_macro.urdf.xacro**
- The **top_structure** was connected to the **base_link** of the husky using and **top_joint**

```
<joint name="${prefix}top_joint" type="fixed">
<origin rpy="0 0 ${M_PI}" xyz="0 0 0.23"/>
<parent link="${prefix}base_link"/>
<child link="top_structure"/>
</joint>
```



Sensor:

- Livox HAP LiDAR for 3D point cloud generation.

Implementation:

- Building Livox-SDK2
- Integrating `livox_laser_simulation_ros2`
- Utilizes the gazebo plugin (`liblivox_simulation.so`) that mimics the LiDAR's output.
- Attaching to the Husky in URDF
 - Modified `husky_macro.urdf.xacro` to include the LiDAR:

```
<xacro:include filename="$(find ros2_livox_simulation)/urdf/HAP.xacro" />
<xacro:HAP name="livox" parent="${prefix}top_structure" topic="HAP">
  <origin xyz="-0.35 0 0.309" rpy="0 0 ${M_PI}"/>
</xacro:HAP>
```



<https://www.livoxtech.com/de/hap>

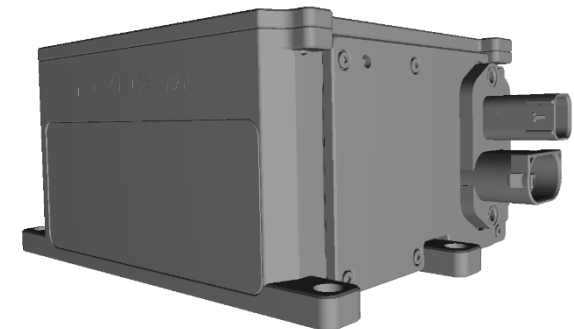


Fig. LIVOX HAP 3d model

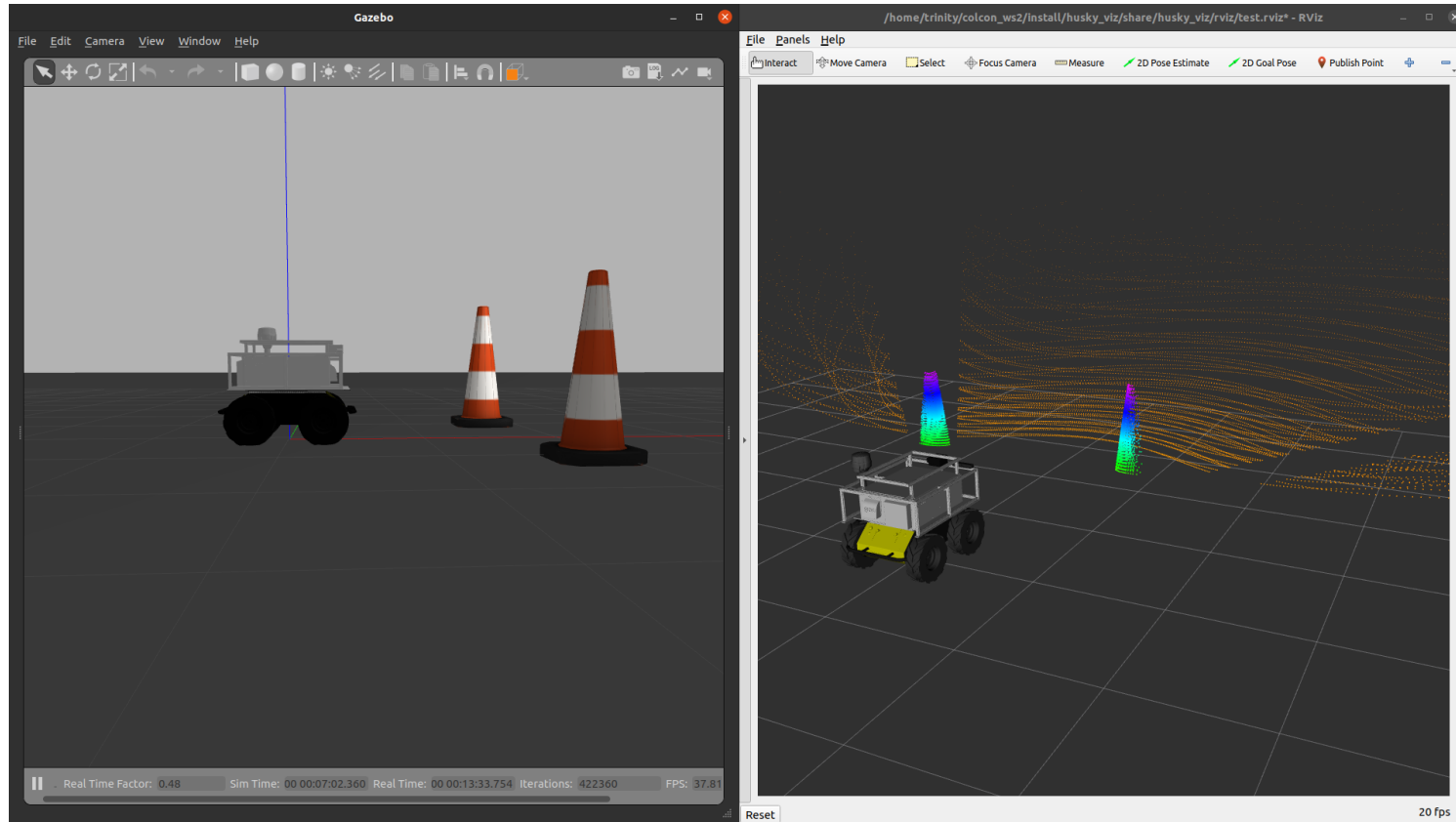


Fig. LIVOX Point Cloud visualization

Sensor: Stereolabs ZED2i for stereo imaging.

Implementation:

- Used ZED ROS2 Wrapper's URDF (zed2.urdf.xacro).
- Modified `husky_macro.urdf.xacro` to attach the camera to the `top_structure`.
- Utilized `libgazebo_ros_camera.so` to simulate stereo camera output in Gazebo.
- Generates virtual left and right images based on the simulated environment.

Published Topics:

- `/zed2_left_raw_camera` (Image)
- `/zed2_right_raw_camera` (Image)

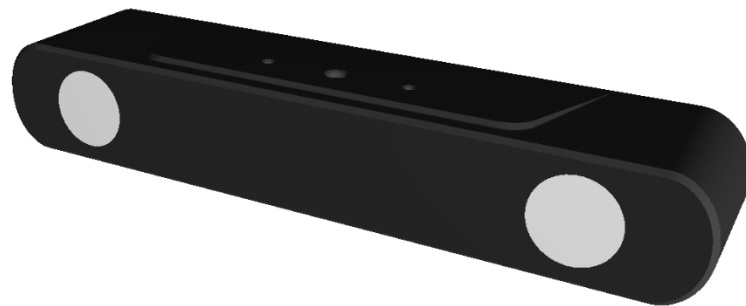


Fig. Stereolabs ZED2i 3d model

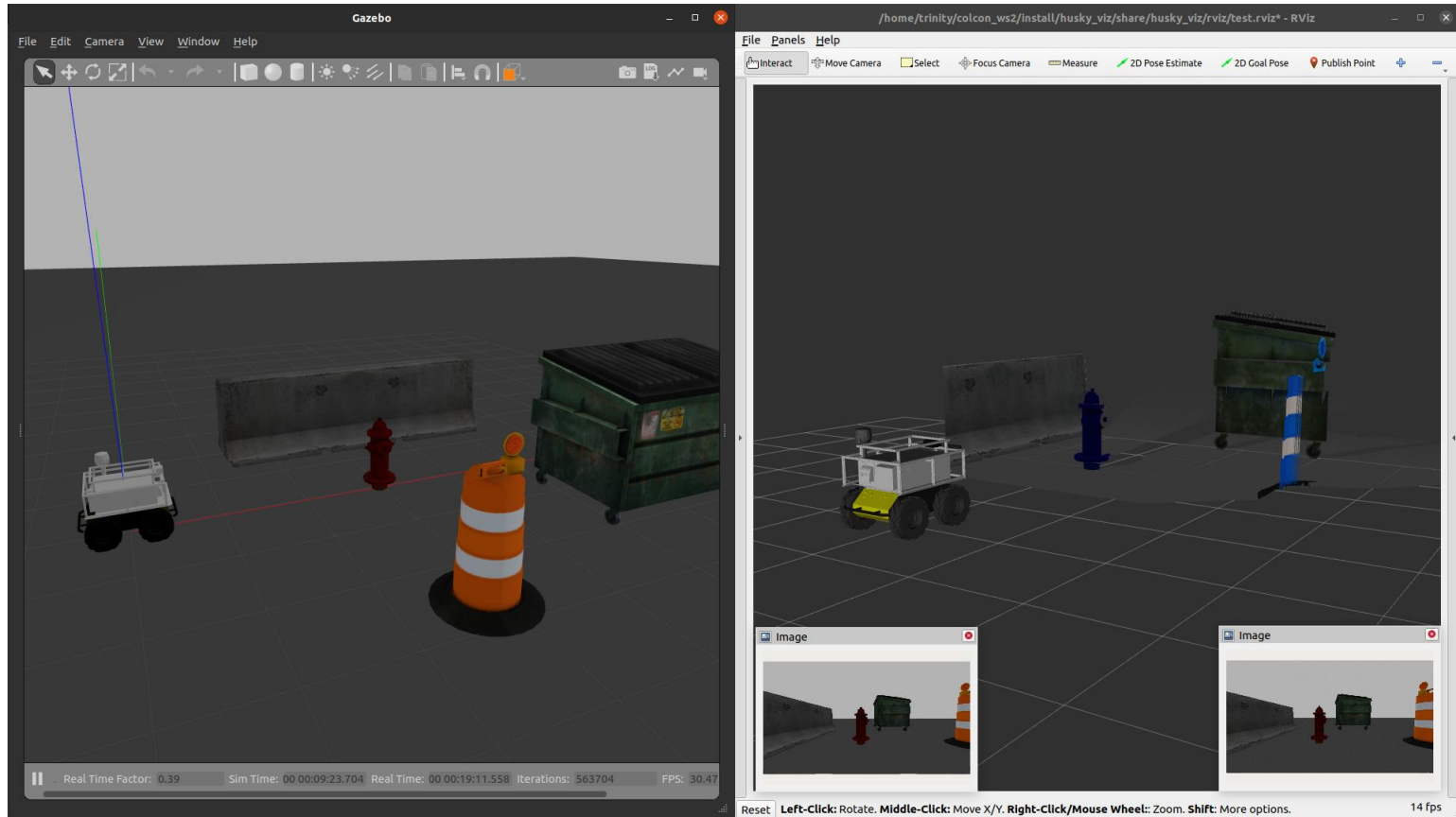


Fig. Depth and stereo images visualization

Sensor: Emlid Reach RS2+ for position tracking.

Implementation:

- Converted STL model and added to URDF
- Simulated with `libgazebo_ros_gps_sensor.so`
- Attached to `top_structure`

Published Topics:

- `/reach_rs2/gps/data`



Fig. Emlid Reach RS2+ GPS



Fig. Emlid Reach RS2+ GPS 3d model

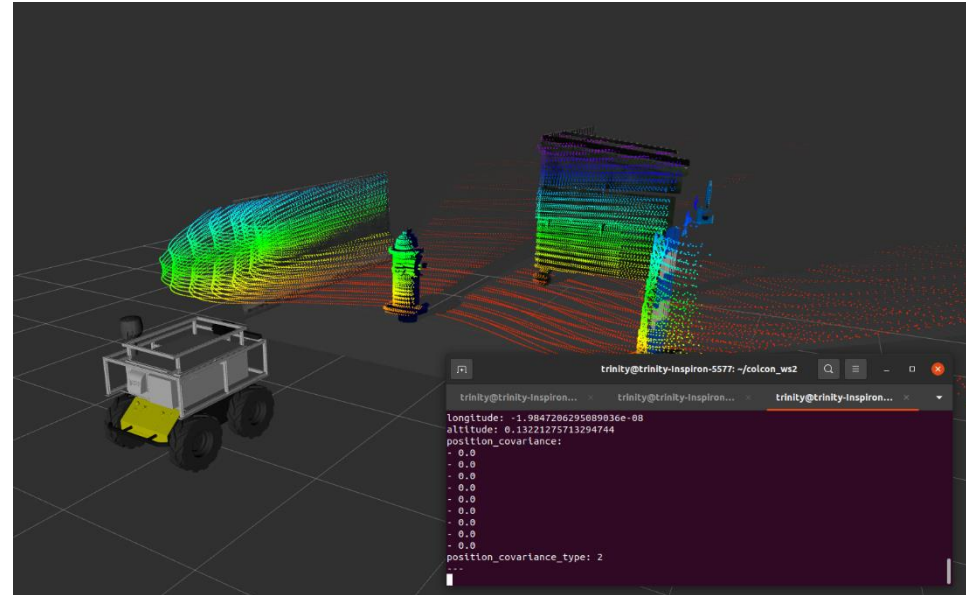


Fig. GPS sensor data visualization



Calculations:

Odometry Distance:

$$dx = end_x - start_x$$

$$dy = end_y - start_y$$

$$Distance = \sqrt{dx^2 + dy^2}$$

GPS Distance (Haversine):

$$a = \sin^2(\Delta lat/2) + \cos(lat1) * \cos(lat2) * \sin^2(\Delta lon/2)$$

$$c = 2 * \operatorname{atan2}(\sqrt{a}, \sqrt{1-a})$$

$$Distance = 6371000 * c$$

Initial Learning Gap with ROS2:

- Limited prior knowledge of ROS2's architecture and tools posed a steep learning curve.

Limited Community Support for ROS2 :

- Fewer resources and community forums were available compared to ROS1, complicating troubleshooting efforts.

Heavy Load on Simulation :

- High computational demands in Gazebo caused lag and instability during simulations.

Troublesome Debugging of Errors:

- Identifying and resolving errors in URDF files or plugin configurations proved complex and time-consuming.

Inconsistent Simulation Loading :

- Simulations occasionally failed to launch or crashed unpredictably, disrupting workflow.

Conclusion:

- Successfully developed a fully functional digital twin with seamlessly integrated sensors.
- Achieved real-time data processing in Gazebo, delivering:
 - Depth and raw images from the camera,
 - 3D point clouds from the Lidar,
 - Precise localization via GPS.
 - Provided comprehensive documentation on GitLab, including step-by-step build instructions and a replication guide.

Future Work:

- Integrate real sensor drivers for hardware testing.
- Expand simulation with complex environments.
- End-to-End Learning for Autonomous Navigation

Digital Twin of Husky

Final Presentation

Thank You For Your Attention!

Manu Benny (244058)
Ijaaz Muhammed Mullamangalam (244754)
Atif Harshad (244570)

